

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №2**  
**по дисциплине «Вычислительная математика»**  
**Тема: решение систем нелинейных уравнений**

Студент гр. 4341

Малышев К.Д.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

### Цель работы.

Целью лабораторной работы являются изучение прямых (метод Гаусса с выбором главного элемента по столбцу) и итерационных (метод Якоби) методов решения систем линейных алгебраических уравнений (СЛАУ), написание программ, реализующих данные методы, и исследование зависимости числа итераций от заданной точности (для метода Якоби), а также исследование обусловленности вычислительных алгоритмов (чувствительности к ошибкам округления и исходным данным).

### Задание.

#### 1) Метод Гаусса

В ходе выполнения работы студенты должны найти решение системы линейных уравнений с  $n$  неизвестными, заданной матрицей коэффициентов  $A$  и вектором свободных членов  $b$ , методом Гаусса. Выполнение работы состоит из следующих этапов:

1) с помощью преподавателя определить систему уравнений, которую нужно решить;

2) для решения системы уравнений разработать программу на языке C, использующую подпрограмму-функцию GAUSS из файла GAUSS.CPP. Данная функция имеет следующие параметры:

- $a$  - матрица коэффициентов системы уравнений размера  $n \times n$ , тип *float* ;
- $b$  - вектор свободных членов размера  $n$ , тип *float* ;
- $x$  - выходной вектор результата решения размера  $n$ , тип *float* ;
- $n$  - размер системы (матрицы  $a$  и вектора свободных членов  $b$ ), тип *int* .

В разрабатываемой программе должна быть описана константа  $n_{\max}$ , равная максимальным размерам используемых матриц и векторов. Функция GAUSS в качестве значения типа *int* возвращает:

- а) 0 - в случае нормального завершения процесса вычисления;
- б) 1 - в случае вырожденности матрицы  $a$ ;
- в) 2 - если  $n < 2$  ;
- г) 3 - если  $n > n_{\max}$  .

## 2)Метод Якоби

В работе студенты должны найти решение системы линейных уравнений с  $n$  неизвестными, заданной матрицей коэффициентов  $A$  и вектором свободных членов  $b$ , методом простых итераций. Выполнение работы включает следующие этапы:

1) с помощью преподавателя определить систему уравнений, которую нужно решить. Привести исходную систему к виду (2.3), пригодному для использования метода простых итераций;

2) задать необходимую точность получения результата (количество знаков мантиссы числа);

3) разработать программу решения задачи на языке C с использованием подпрограммы-функции MITER из файла MITER.CPP. Функция MITER имеет следующие параметры:

-  $a$  - матрица коэффициентов преобразованной к виду (2.3) системы уравнений размера  $n \times n$  , тип *float* ;

-  $b$  - вектор свободных членов преобразованной системы размера  $n$  , тип *float* ;

-  $x$  - полученный в результате проведения итераций вектор решения размера  $n$  , тип *float* ;

-  $n$  - размер системы уравнений, тип *int* ;

- *ntch* - количество знаков после запятой в мантиссе результата, остающихся после округления, тип *int* ,  $0 \leq ntch \leq 6$  ;

- *it* - выходной параметр, равный количеству произведенных итераций, тип *int*.

В качестве значения функции типа *int* возвращается одно из следующих значений:

- а) 0 - все нормально, получено решение  $x$  ;
- б) 1 - не выполняются условия сходимости итерационного процесса;
- в) 2 - размер  $n < 2$  ;
- г) 3 - значение  $n > n_{\max}$  .

Константа  $n_{\max}$  должна быть задана при разработке головной программы аналогично тому, как это делается при выполнении лабораторной работы № 11;

4) произвести вычисления с использованием разработанной программы и построить график зависимости числа итераций от задаваемой точности.

### **Описание методов.**

Суть метода исключения по главным элементам (метод Гаусса) заключается в приведении исходной системы линейных уравнений  $Ax=b$  к треугольному виду. Процесс приведения системы к такому виду называется прямым ходом, а последующее нахождение неизвестных  $x_1, \dots, x_n$  — обратным ходом метода Гаусса.

На каждом шаге прямого хода в очередном столбце находится наибольший по абсолютной величине коэффициент  $a_{kj}$ . Для исключения  $x_j$  из  $i$ -го уравнения ( $i \neq k$ ) необходимо умножить  $k$ -е уравнение на коэффициент  $a_{ij}/a_{kj}$  и вычесть его из  $i$ -го уравнения. Этот процесс повторяется для всех оставшихся неизвестных. Если перед исключением

неизвестной найденный главный элемент окажется равным нулю, это свидетельствует о вырожденности матрицы (определитель равен нулю).

В данной работе используется частичный выбор главного элемента, при котором на каждом шаге выбирается максимальный по модулю элемент в текущем столбце и производится перестановка строк. Это позволяет уменьшить накопление вычислительных ошибок.

Метод Якоби является итерационным методом решения системы линейных алгебраических уравнений вида:

$$Ax = b$$

где  $A$  — квадратная матрица коэффициентов,  $x$  — вектор неизвестных,  $b$  — вектор свободных членов.

Основная идея метода заключается в последовательном уточнении приближений к решению.

Каждая компонента вектора  $x$  на новой итерации вычисляется независимо от других, используя значения предыдущей итерации.

Для этого каждое уравнение системы переписывается относительно соответствующей переменной  $x_i$ :

$$x_i = \frac{1}{a_{ii}} (b_i - \sum_{j \neq i} a_{ij} x_j)$$

Пусть задано начальное приближение  $x(0)$ . Тогда на  $k$ -й итерации новое приближение вычисляется по формуле:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} (b_i - \sum_{j \neq i} a_{ij} x_j^{(k)})$$

Таким образом, на каждой итерации формируется новый вектор  $x^{(k+1)}$ , полностью зависящий от предыдущего вектора  $x^{(k)}$ .

Важно отметить, что все компоненты нового вектора вычисляются параллельно, без использования уже обновлённых значений.

Итерационный процесс продолжается до достижения заданной точности. Обычно используется условие:

$$\|x^{(k+1)} - x^{(k)}\| < \varepsilon$$

где  $\varepsilon$  — заданная погрешность, а норма чаще всего берётся как максимум:

$$\|x\|_{\infty} = \max |x_i|$$

Это означает, что метод останавливается, когда изменения между соседними итерациями становятся достаточно малыми.

Метод Якоби сходится не для всех матриц. Достаточным условием сходимости является диагональное преобладание:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Это означает, что в каждой строке матрицы модуль диагонального элемента должен быть больше суммы модулей остальных элементов.

При выполнении этого условия итерационный процесс гарантированно сходится к точному решению системы.

### **Разработка программы.**

Программа, реализующая метод Гаусса с выбором главного элемента, была написана на языке программирования C++.

В соответствии с заданием, была реализована основная функция `int GAUSS(double a[nmax][nmax], double b[nmax], double x[nmax], int n)` для решения системы уравнений. Функция принимает следующие параметры:

- $a$  — матрица коэффициентов системы уравнений размера  $n \times n$ ;
- $b$  — вектор свободных членов размера  $n$ ;

- $x$  — выходной вектор результата решения размера  $n$ ;
- $n$  — размер системы (матрицы  $a$  и вектора  $b$ ).

В программе была описана константа  $nmax = 10000$ , равная максимальным размерам используемых матриц и векторов. Функция GAUSS возвращает целочисленные коды завершения:

- 0 — в случае нормального завершения процесса вычисления;
- 1 — в случае вырожденности матрицы  $a$ ;
- 2 — если  $n < 2$ ;
- 3 — если  $n > nmax$ .

Для проведения автоматического тестирования и проверки алгоритма на матрицах различных размерностей были дополнительно реализованы следующие функции:

`void randomMatrix(double A[nmax][nmax], int n)` — генерирует матрицу случайных чисел в заданном диапазоне для стресс-тестирования алгоритма на больших размерностях (до 10000).

`void hilbertMatrix(double A[nmax][nmax], int n)` — генерирует плохо обусловленную матрицу Гильберта, элементы которой вычисляются по формуле  $A_{ij} = 1 / (i + j + 1.0)$ . Эта функция необходима для исследования работы метода на плохо обусловленных задачах.

`double residualNorm(double A[nmax][nmax], double x[nmax], double b[nmax], int n)` — вычисляет евклидову

норму вектора невязки по формуле  $\sqrt{\sum_{i=1}^n (Ax - b)_i^2}$  для оценки точности полученного решения.

Программа, реализующая метод Якоби была написана на языке программирования C++.

В соответствии с заданием, была реализована основная функция

```
int JACOBI(const vector<double> &A, const
vector<double> &b, vector<double> &x, int n, int
max_iter, double eps)
```

Функция принимает следующие параметры:

- A — матрица коэффициентов системы уравнений размера  $n \times n$ , представленная в виде одномерного массива;
- b — вектор свободных членов размера  $n$ ;
- x — выходной вектор результата решения размера  $n$ ;
- n — размер системы;
- max\_iter — максимальное число итераций;
- eps — заданная точность.

Функция JACOBI возвращает целочисленные коды завершения:

- 0 — в случае успешной сходимости метода;
- 1 — если метод не сошёлся за заданное число итераций;
- 2 — если метод неприменим (нулевой элемент на диагонали матрицы).

```
void diagonalDominantMatrix(vector<double> &A,
int n)
```

Данная функция генерирует матрицу с диагональным преобладанием.

Элементы матрицы заполняются случайными значениями, после чего диагональные элементы увеличиваются таким образом, чтобы их



абсолютное значение превышало сумму модулей остальных элементов строки.

```
double residualNorm(const vector<double> &A,  
const vector<double> &x, const vector<double> &b,  
int n)
```

Функция вычисляет евклидову норму вектора невязки решения системы линейных уравнений по формуле:

$$|Ax - b|$$

Данная величина позволяет оценить точность полученного решения и корректность работы алгоритма.

### **Результаты вычислений метода Гаусса.**

Тест №1: Проверка на системе малого порядка

Для проверки корректности работы прямого и обратного хода метода Гаусса была решена система уравнений порядка  $n = 3$

Матрица A:

2 1 -1

-3 -1 2

-2 1 2

| Неизвестная | Полученное значение | Ожидаемое значение |
|-------------|---------------------|--------------------|
| x[0]        | 2.0000000000        | 2                  |
| x[1]        | 3.0000000000        | 3                  |
| x[2]        | -1.0000000000       | -1                 |

Вектор B:

8

-11

-3

Программа успешно нашла точные значения корней, что подтверждает корректность реализации алгоритма.

#### Тест №2: Автотест на больших размерностях

В ходе теста исследовалась зависимость времени вычисления от размерности системы  $n$  и оценивалась норма невязки.

| Размерность | Время выполнения(сек) | Невязка(евклидова норма) |
|-------------|-----------------------|--------------------------|
| 1000        | 0.6378                | $2 * 10^{-10}$           |
| 5000        | 81.8418               | $7.6 * 10^{-9}$          |
| 10000       | 662.0114              | $3.9 * 10^{-8}$          |

Наблюдается кубическая зависимость времени вычисления от порядка матрицы, что соответствует теоретической сложности метода Гаусса  $O(n^3)$ . Увеличение размерности в 10 раз (со 1000 до 10000) привело к росту времени выполнения примерно в 1000 раз. Невязка остается в пределах допустимых погрешностей для типа `double`.

#### Тест №3: Матрица Гильберта и исследование обусловленности

Для исследования влияния обусловленности на точность решения была выбрана матрица Гильберта порядка  $n = 10$ .

Невязка: 0.0000000000

Решения (x):

0.9999999991      1.0000000776      0.9999983543      1.0000149063  
0.9999291061    1.0001944260    0.9996816350    1.0003071486    0.9998389815  
1.0000353661

Несмотря на то что невязка практически равна нулю, полученные значения  $x$  имеют отклонения от эталонных единиц уже в 5–7 знаке после запятой. Это объясняется тем, что матрица Гильберта является плохо обусловленной

### Результаты вычислений метода Якоби.

Тест №1: Проверка на системе малого порядка

Для проверки корректности работы метода Якоби была решена система уравнений порядка  $n = 3$

Матрица  $A$ :

5 1 1

1 4 -1

2 -1 5

Вектор  $B$ :

6 10 -5

| Неизвестная | Полученное значение | Ожидаемое значение |
|-------------|---------------------|--------------------|
| $x[0]$      | 0.9999999955        | 1                  |
| $x[1]$      | 2.0000000052        | 2                  |
| $x[2]$      | -0.9999999942       | -1                 |

$$|Ax - b| = 0.0000000214$$

Тест №2: Автотест на больших размерностях

В ходе теста исследовалась зависимость времени вычисления от размерности системы  $n$  и оценивалась норма невязки.

| Размерность | Время | Невязка |
|-------------|-------|---------|
|-------------|-------|---------|

|       | выполнения(сек) |              |
|-------|-----------------|--------------|
| 1000  | 0.0219009000    | 0.0000062935 |
| 5000  | 0.4896292000    | 0.0000159355 |
| 10000 | 1.9467920000    | 0.0000058875 |

### **Выводы.**

В ходе выполнения лабораторной работы были изучены и реализованы методы решения СЛАУ: метод Гаусса и метод Якоби. Метод Якоби работает за гораздо меньшее число итераций, однако даёт большую погрешность и подходит только для матриц с диагональным преобладанием.

## Приложение А

### Исходные коды программ

#### Метод Гаусса

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <random>
#include <chrono>

using namespace std;

const int nmax = 10000;

// ===== GAUSS =====
int GAUSS(double a[nmax][nmax], double b[nmax], double x[nmax], int n)
{
    int i, j, k, imax;
    double amax, t, factor, sum;

    if (n < 2) return 2;
    if (n > nmax) return 3;

    // Прямой ход
    for (k = 0; k < n - 1; k++) {
        imax = k;
        amax = fabs(a[k][k]);

        for (i = k + 1; i < n; i++) {
            if (fabs(a[i][k]) > amax) {
                amax = fabs(a[i][k]);
                imax = i;
            }
        }

        if (amax < 1e-12) return 1;

        // swap строк
        if (imax != k) {
            for (j = k; j < n; j++) { // Оптимизация: меняем местами
                t = a[k][j];
                a[k][j] = a[imax][j];
                a[imax][j] = t;
            }
            t = b[k];
            b[k] = b[imax];
            b[imax] = t;
        }

        // исключение
        for (i = k + 1; i < n; i++) {
            factor = a[i][k] / a[k][k];
            a[i][k] = 0.0;
```

```

        for (j = k + 1; j < n; j++) {
            a[i][j] -= factor * a[k][j];
        }
        b[i] -= factor * b[k];
    }
}

if (fabs(a[n - 1][n - 1]) < 1e-12) return 1;

// Обратный ход
for (i = n - 1; i >= 0; i--) {
    sum = 0.0;
    for (j = i + 1; j < n; j++) {
        sum += a[i][j] * x[j];
    }
    x[i] = (b[i] - sum) / a[i][i];
}

return 0;
}

// ===== СЛУЧАЙНАЯ =====
void randomMatrix(double A[nmax][nmax], int n) {
    mt19937 gen(42);
    uniform_real_distribution<> dis(-10.0, 10.0);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            A[i][j] = dis(gen);
}

// ===== ГИЛЬБЕРТ =====
void hilbertMatrix(double A[nmax][nmax], int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            A[i][j] = 1.0 / (i + j + 1.0);
}

// ===== НОРМА =====
double residualNorm(double A[nmax][nmax], double x[nmax], double
b[nmax], int n) {
    double norm = 0.0;
    for (int i = 0; i < n; i++) {
        double sum = 0;
        for (int j = 0; j < n; j++) {
            sum += A[i][j] * x[j];
        }
        norm += (sum - b[i]) * (sum - b[i]);
    }
    return sqrt(norm);
}

// ===== MAIN =====
int main() {
    cout << fixed << setprecision(10);

    // ВЫДЕЛЕНИЕ ПАМЯТИ В КУЧЕ (Heap) ДЛЯ ПРЕДОТВРАЩЕНИЯ STACK OVERFLOW

```

```

auto* A = new double[nmax][nmax];
auto* A_copy = new double[nmax][nmax];
auto* b = new double[nmax];
auto* b_copy = new double[nmax];
auto* x = new double[nmax];
auto* x_true = new double[nmax];

while (true) {
    cout << "\n1. Решить систему (ручной ввод)\n";
    cout << "2. Автотест (большие размерности)\n";
    cout << "3. Тест матрицы Гильберта\n";
    cout << "0. Выход\n";
    cout << ">> ";

    int choice;
    cin >> choice;

    if (choice == 0) break;

    if (choice == 1) {
        int n;
        cout << "n = ";
        cin >> n;

        cout << "Введите матрицу (" << n << "x" << n << "):\n";
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                cin >> A[i][j];

        cout << "Введите b:\n";
        for (int i = 0; i < n; i++)
            cin >> b[i];

        int err = GAUSS(A, b, x, n);

        if (err == 0) {
            for (int i = 0; i < n; i++)
                cout << "x[" << i << "] = " << x[i] << "\n";
        } else {
            cout << "Ошибка метода Гаусса: Код " << err << "\n";
        }
    }

    if (choice == 2) {
        // Размеры требуемые по заданию
        int sizes[] = {1000, 5000, 10000};

        for (int s = 0; s < 3; s++) {
            int n = sizes[s];
            cout << "\n=== n = " << n << " ===\n";
            if (n == 10000) cout << "(Подождите, вычисление может занять несколько минут...)\n";

            randomMatrix(A, n);

            for (int i = 0; i < n; i++) {
                x_true[i] = 1.0;
                b[i] = 0;
            }
        }
    }
}

```

```

    }

    // Создаем вектор b = A * x_true
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            b[i] += A[i][j] * x_true[j];

    // Делаем копии исходных данных для вычисления невязки!
    for (int i = 0; i < n; i++) {
        b_copy[i] = b[i];
        for (int j = 0; j < n; j++)
            A_copy[i][j] = A[i][j];
    }

    auto start = chrono::high_resolution_clock::now();
    int err = GAUSS(A, b, x, n);
    auto end = chrono::high_resolution_clock::now();

    if (err != 0) {
        cout << "Ошибка вырождения\n";
        continue;
    }

    cout << "Невязка: " << residualNorm(A_copy, x, b_copy,
n) << "\n";

    cout << "Время: " << chrono::duration<double>(end -
start).count() << " сек\n";
    }
}

if (choice == 3) {
    cout << "\n=== Матрица Гильберта ===\n";
    int n = 10;
    hilbertMatrix(A, n);

    for (int i = 0; i < n; i++) {
        b[i] = 0;
        for (int j = 0; j < n; j++)
            b[i] += A[i][j]; // x_true = [1, 1... 1]
    }

    // Обязательно копируем!
    for (int i = 0; i < n; i++) {
        b_copy[i] = b[i];
        for (int j = 0; j < n; j++)
            A_copy[i][j] = A[i][j];
    }

    int err = GAUSS(A, b, x, n);

    if (err == 0) {
        // Считаем невязку по копиям
        cout << "Невязка: " << residualNorm(A_copy, x, b_copy,
n) << "\n";

        cout << "Решения (x):\n";
        for(int i=0; i<n; i++) cout << x[i] << " ";
        cout << "\n";
    } else {

```



```

        cout << "Ошибка\n";
    }
}

// Очистка памяти
delete[] A; delete[] A_copy;
delete[] b; delete[] b_copy;
delete[] x; delete[] x_true;

return 0;
}

```

## Метод Якоби

```

#include <iostream>
#include <iomanip>
#include <cmath>
#include <vector>
#include <random>
#include <chrono>

using namespace std;

const int NMAX = 10000; // глобальная константа

int JACOBI(const vector<double> &A, const vector<double> &b,
           vector<double> &x, int n, int max_iter,
           double eps, int &it) {

    // Проверка размера
    if (n < 2) return 2;
    if (n > NMAX) return 3;

    // Проверка условий сходимости (диагональное преобладание)
    for (int i = 0; i < n; ++i) {
        double diag = fabs(A[i * n + i]);
        if (diag < 1e-12) return 1; // нулевой диагональный элемент
        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (j != i) sum += fabs(A[i * n + j]);
        }
        if (diag <= sum) return 1; // нет строгого диагонального
        преобладания
    }

    x.assign(n, 0.0);
    vector<double> x_new(n);

    for (it = 0; it < max_iter; ++it) {
        for (int i = 0; i < n; ++i) {
            double diag = A[i * n + i];
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                if (j != i) sum += A[i * n + j] * x[j];
            }

```

```

        x_new[i] = (b[i] - sum) / diag;
    }

    double max_diff = 0.0;
    for (int i = 0; i < n; ++i) {
        max_diff = max(max_diff, fabs(x_new[i] - x[i]));
        x[i] = x_new[i];
    }

    if (max_diff < eps) return 0;
}

return 1; // не сошлось за max_iter - тоже считаем нарушением
сходимости
}

void diagonalDominantMatrix(vector<double> &A, int n) {
    // Используем фиксированный seed для воспроизводимости
    mt19937 gen(42);
    uniform_real_distribution<double> dis(-5.0, 5.0);

    for(int i = 0; i < n; ++i) {
        double sum = 0.0;
        for(int j = 0; j < n; ++j) {
            if (i != j) {
                double val = dis(gen);
                A[i * n + j] = val;
                sum += fabs(val);
            }
        }
        // Сделаем диагональный элемент на 1 больше суммы модулей
        остальных
        A[i * n + i] = sum + fabs(dis(gen)) + 1.0;
    }
}

double residualNorm(const vector<double> &A, const vector<double> &x,
                    const vector<double> &b, int n) {
    double norm_sq = 0.0;
    for(int i = 0; i < n; ++i) {
        double sum = 0.0;
        for(int j = 0; j < n; ++j) {
            sum += A[i * n + j] * x[j];
        }
        double diff = sum - b[i];
        norm_sq += diff * diff;
    }
    return sqrt(norm_sq);
}

int main() {
    cout << fixed << setprecision(10);

    while(true) {
        cout << "\n=== Меню ===\n";
        cout << "1. Решить систему (ручной ввод)\n";
        cout << "2. Автотест (различные размерности)\n";
        cout << "3. График: зависимость итераций от точности\n";
    }
}

```

```

cout << "0. Выход\n";
cout << "Ваш выбор: ";

int choice;
if(!(cin >> choice)) {
    cout << "Ошибка ввода\n";
    return 0;
}

if(choice == 0) break;

if(choice == 1) {
    int n;
    cout << "Введите размер системы n: ";
    cin >> n;
    if(n <= 0) {
        cout << "Некорректный размер\n";
        continue;
    }
    vector<double> A(n * n);
    vector<double> b(n), x(n);

    cout << "Введите матрицу A (" << n << "x" << n << "):\n";
    for(int i = 0; i < n; ++i) {
        for(int j = 0; j < n; ++j) {
            cin >> A[i * n + j];
        }
    }

    cout << "Введите вектор b:\n";
    for(int i = 0; i < n; ++i) {
        cin >> b[i];
    }

    int max_iter = 10000;
    double eps = 1e-8;
    int it;
    int err = JACOBI(A, b, x, n, max_iter, eps, it);

    if(err == 0) {
        cout << "Решение (x):\n";
        for(int i = 0; i < n; ++i) {
            cout << "x[" << i << "] = " << x[i] << "\n";
        }
        // Вычисляем невязку Ax - b
        double res = residualNorm(A, x, b, n);
        cout << "Невязка ||Ax - b|| = " << res << "\n";
    }
    else if(err == 1) {
        cout << "Метод Якоби не сошёлся за " << max_iter << "
итераций\n";
    }
    else if(err == 2) {
        cout << "Ошибка: n < 2\n";
    }
    else if(err == 3) {
        cout << "Ошибка: n > NMAX\n";
    }
}

```

```

        else {
            cout << "Ошибка: диагональный элемент нулевой или нет
сходимости\n";
        }
    }

    if(choice == 2) {
        // Размеры автотеста:
        vector<int> sizes = {1000, 5000, 10000};
        cout << "\n=== Автотест метода Якоби ===\n";

        for(int n : sizes) {
            cout << "\n--- n = " << n << " ---\n";

            vector<double> A(n * n), b(n), x(n);
            // Генерируем диагонально-преобладающую матрицу A
            diagonalDominantMatrix(A, n);

            // Точное решение x_true = (1,1,...,1)
            // Собираем вектор правой части b = A * x_true
            for(int i = 0; i < n; ++i) {
                b[i] = 0.0;
                for(int j = 0; j < n; ++j) {
                    b[i] += A[i * n + j] * 1.0;
                }
            }

            // Сохраним копию b для подсчёта невязки
            vector<double> b_copy = b;
            int it;
            auto start = chrono::high_resolution_clock::now();
            int err = JACOBI(A, b, x, n, 10000, 1e-8, it);
            auto end = chrono::high_resolution_clock::now();

            if(err != 0) {
                cout << "Метод Якоби не сошёлся (код " << err <<
")\n";
                continue;
            }
            double res = residualNorm(A, x, b_copy, n);
            cout << "Итераций: " << it << "\n";
            cout << "Невязка ||Ax - b|| = " << res << "\n";
            cout << "Время: "
                << chrono::duration<double>(end - start).count()
                << " сек\n";
        }
    }

    if(choice == 3) {
        int n = 10000;
        cout << "\n=== Зависимость числа итераций от точности
===\n";

        vector<double> A(n * n), b(n), x(n);

        diagonalDominantMatrix(A, n);

        // x_true = (1,...,1)
        for(int i = 0; i < n; i++) {

```

```

        b[i] = 0;
        for(int j = 0; j < n; j++) {
            b[i] += A[i * n + j];
        }
    }

    cout << "k\t eps\t\t итерации\n";

    for(int k = 2; k <= 10; k++) {
        double eps = pow(10.0, -k);

        int it;
        int err = JACOBI(A, b, x, n, 10000, eps, it);

        if(err == 0) {
            cout << k << "\t" << eps << "\t" << it << "\n";
        } else {
            cout << k << "\t" << eps << "\t" << "не сошёлся\n";
        }
    }
}

return 0;
}

```